

C# PROGRAMMING

(330)

NATIONALS 2022

Production:

Program 1: Substitution Cipher Tool _____ (530 points)

TOTAL POINTS _____ (530 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Contestant must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

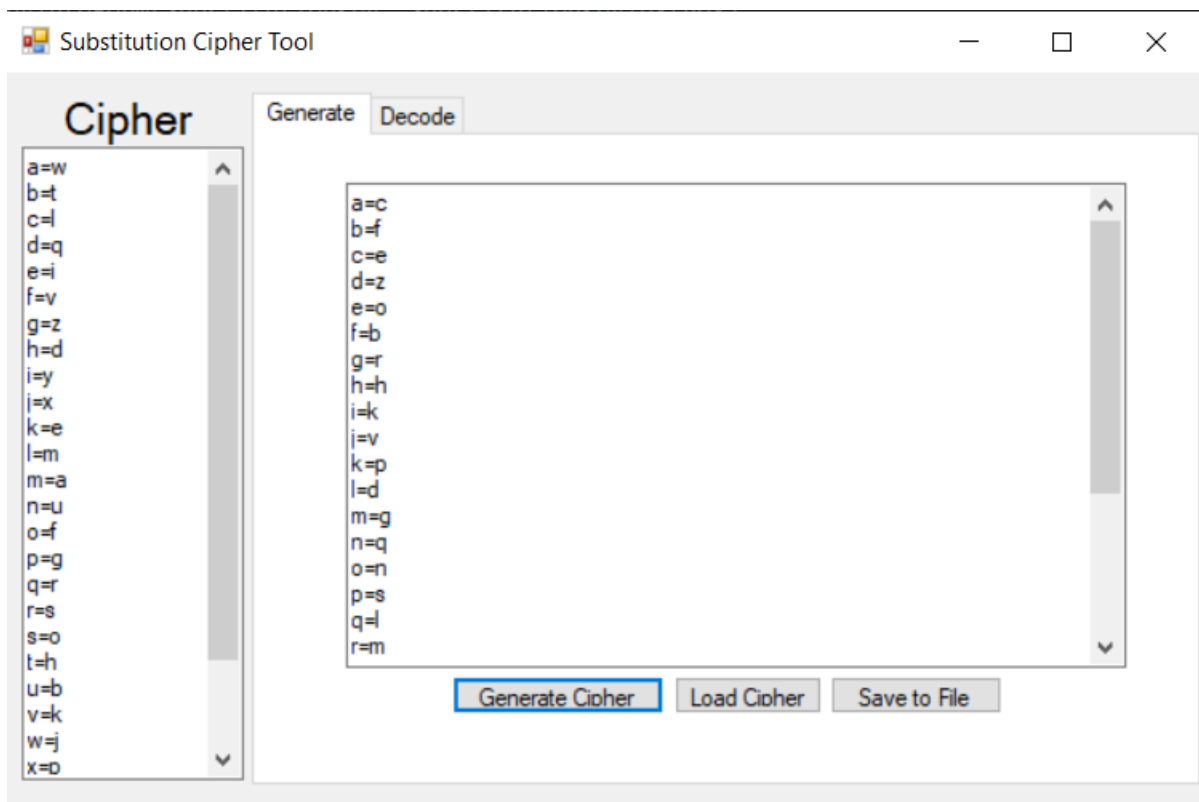
NOTE TO JUDGES:

Use the provided grader files to test the application.

Substitution Cipher Tool

In this exercise you will create a C# Windows Form Application that generates substitution ciphers and uses them to decode encoded text. A substitution cipher encrypts text by replacing each letter in a message with a different letter. Which letters map to other letters is determined by the cipher key. For example, the string “hello” could be encrypted as “wkppv” if the cipher key contains the following mappings: h=w, e=k, l=p, and o=y. For this exam we will ignore non-alphabetical characters and spaces (“hello!” is encrypted as “wkppy!” using the key above) and ignore casing (“h” and “H” both encrypt as “w”).

The substitution cipher tool should perform two main tasks. First, the tool should be capable of randomly generating a substitution cipher key for the entire alphabet. The user should also be able to load this cipher into the program and save the cipher to the text file *SubstitutionKey.txt*. Second, the tool should be capable of decrypting encrypted text loaded from the text file *EncodedTexts.txt*.



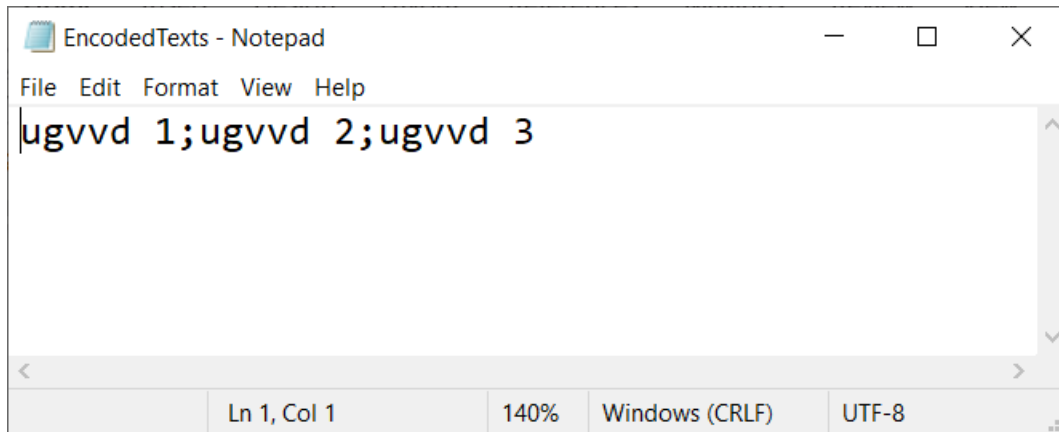
Requirements:

1. You must create a C# Windows Form Application named CS _330_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CS_123_45_6789_0000.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. The main form must not have minimize or maximize buttons in the Form Handle.
4. The main form must have text “Substitution Cipher Tool” for its title. The form must start in the center of the screen and must have a Fixed Dialog setting so the size cannot be adjusted.
5. The main form must contain the following items:
 - a. One large textbox for the loaded cipher key, with the following properties
 - i. Name: “SubstitutionKeyTextBox”
 - ii. Size: Width = 147, Height = 388
 - iii. Multiline: true
 - iv. Readonly: true
 - b. One label above the textbox with the loaded cipher key
 - i. Name: “CipherLabel”
 - ii. Font Size: 16pt
 - iii. Text: “Cipher”
 - c. One large TabControl, containing two tabs
 - i. Name: “CipherToolsTabControl”
 - ii. Size: Width = 634, Height = 426
 - iii. First tab
 1. Name: “GenerateCipherTab”
 2. Text: “Generate”
 - iv. Second tab
 1. Name: “DecodeCiphersTab”
 2. Text: “Decode”
 - d. In the “Generate” Tab
 - i. One large text box to display the generated cipher key
 1. Name: “GenerateKeyTextbox”
 2. Size: Width = 518, Height = 297
 3. ScrollBars: “Vertical”
 4. Multiline: True
 5. Readonly: True
 - ii. Three buttons, one for each operation
 1. “Generate Cipher” Button
 - a. Name: “GenerateCipherButton”
 - b. Text: “Generate Cipher”
 2. “Load Cipher” Button

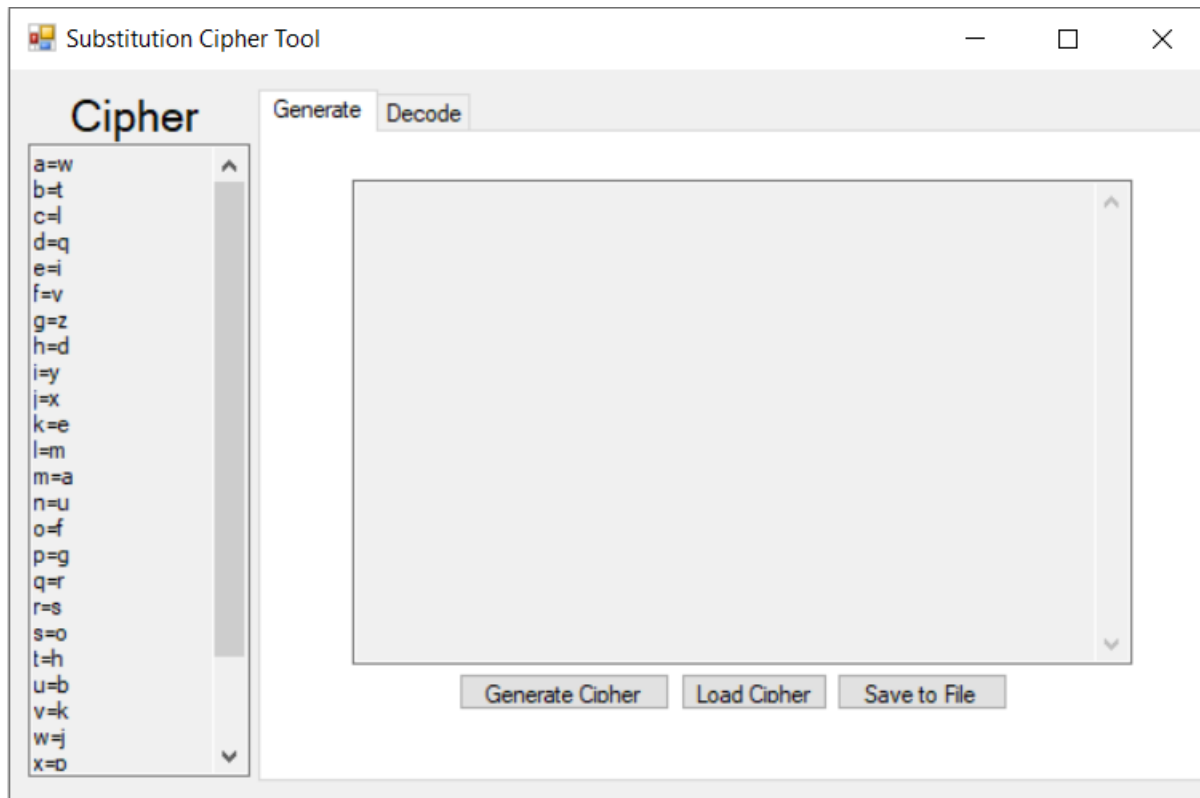
- a. Name: "LoadCipherButton"
 - b. Text: "Load Cipher"
 - 3. "Save to File" Button
 - a. Name: "SaveToFileButton"
 - b. Text: "Save to File"
 - e. In the "Decode" Tab
 - i. One large text box on the left to display the encoded texts
 - 1. Name: "EncodedTextsTextBox"
 - 2. Size: Width = 271, Height = 315
 - 3. Multiline: True
 - 4. ScrollBars: "Vertical"
 - ii. One large text box on the right to display the decoded texts
 - 1. Name: "DecodedTextsTextBox"
 - 2. Size: Width = 271, Height = 315
 - 3. Multiline: True
 - 4. ReadOnly: True
 - 5. ScrollBars: "Vertical"
 - iii. One button
 - 1. Name: "DecodeTextButton"
 - 2. Text: "Decode Text"
- 6. When the program first loads, the main window needs the following:
 - a. The SubstitutionKeyTextBox should be populated with the cipher key loaded from the text file *SubstitutionKey.txt*. Each mapping should be on its own line
 - i. The text file should have the cipher key as one string of characters in the first line. An example text file could contain the string "wtlqivzdyxemaufgrsohbkjpcn", where $a = w$, $b = t$, ..., $z = n$.
 - ii. If the text file is not formatted correctly, display a MessageBox with an error message and exit the application (see pictures section for reference)
 - b. The EncodedTextsTextBox should be populated with the encoded texts loaded from the text file *EncodedTexts.txt*. Each encoded text should be on its own line
 - i. The text file should have the encoded texts as one string on the first line of the file. Texts are separated by semi-colons. An example text file could contain the string "This is test 1;This is test 2;This is test 3", which contains three texts.
 - ii. If the text file is not formatted correctly, display a MessageBox with an error message and exit the application (see pictures section for reference)
- 7. The "Generate Cipher" button in the "Generate" tab must use a click event called "GenerateCipherButton_Click" which performs the following functions:
 - a. Randomly generate a complete substitution cipher key for every letter in the alphabet.
 - b. Update the text of the "GenerateKey" text box with the newly generated cipher key

8. The “Load Cipher” button in the “Generate” tab must use a click event called “LoadCipherButton_Click” which performs the following functions:
 - a. Replace the current cipher key with the cipher key in the “GenerateKey” text box
 - i. This will likely be as simple as updating whatever property/variable you are storing the cipher key with a new value.
 - b. Update the text of the “SubstitutionKey” text box with the cipher key in the “GenerateKey” text box.
 - c. If the “GenerateKey” text box has no text, then do nothing.
9. The “Save to File” button in the “Generate” tab must use a click event called “SaveToFileButton_Click” which performs the following functions:
 - a. Convert the current cipher key to a string of the same form as specified in instruction 6ai above.
 - b. Overwrite the contents of the text file *SubstitutionKey.txt* with the converted cipher key string.
10. The “Decode Text” button in the “Decode” tab must use a click event called “DecodeTextButton_Click” which performs the following functions:
 - a. Using the loaded cipher key, decode each string by substituting every alphabetical character in the encoded string with its corresponding letter in the key to produce the decoded string.
 - b. Update the text of the “DecodedTexts” text box with the newly decoded strings.

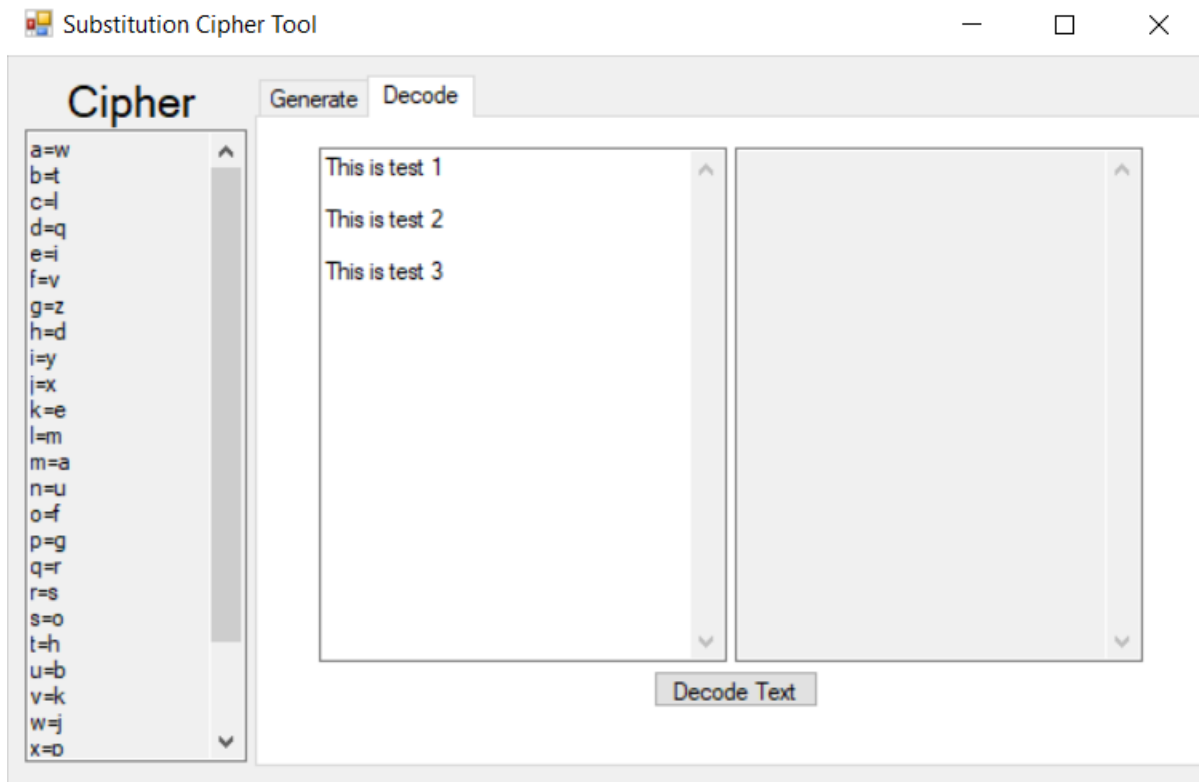
Sample Pictures:



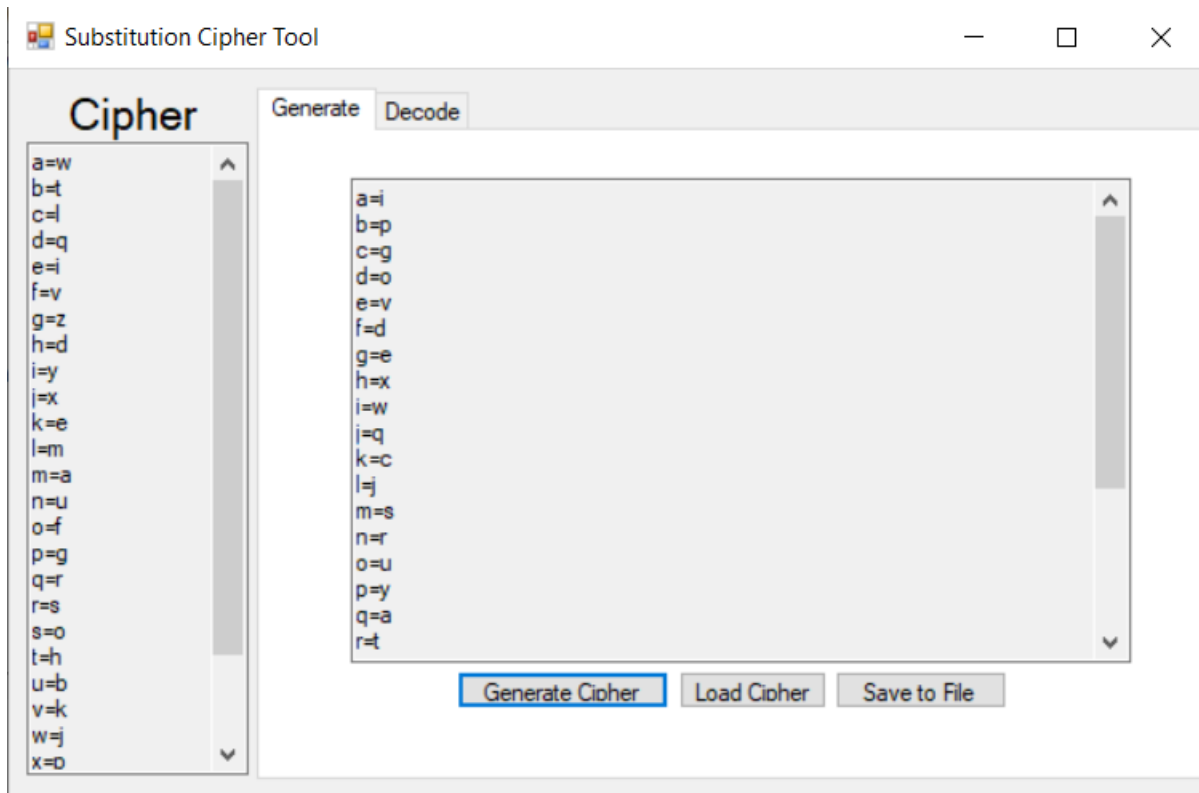
1: An example of what the EncodedTexts.txt file should contain.



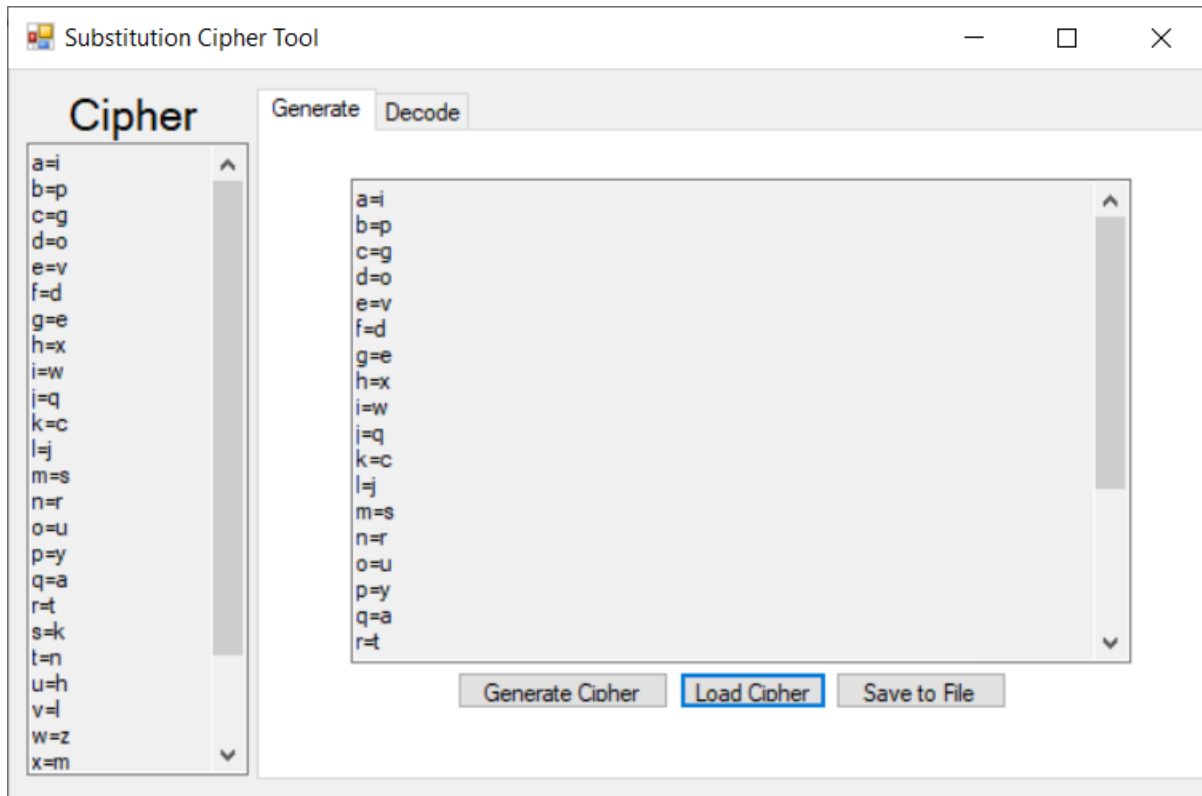
2: What the "Generate" tab of the cipher tool should look like when the program starts.



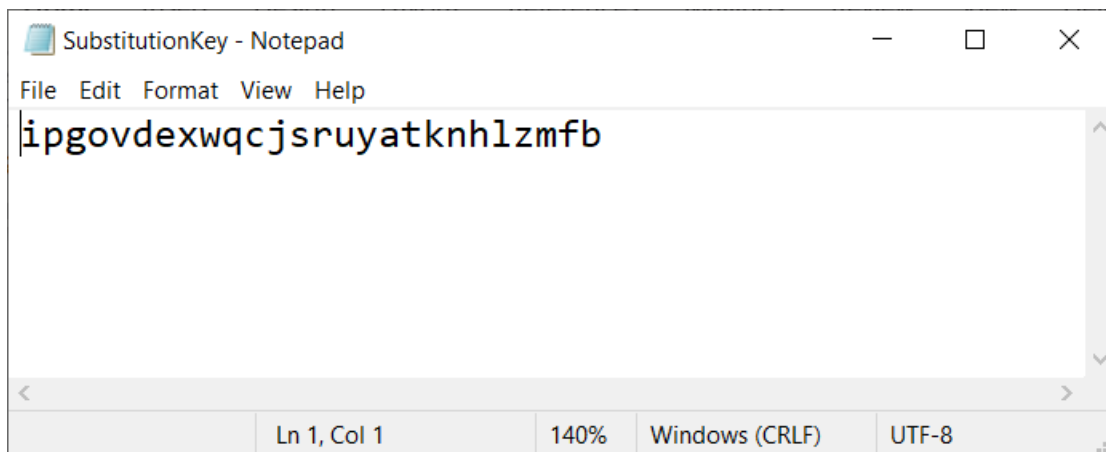
3: What the "Decode" tab of the cipher tool should look like when the program starts.



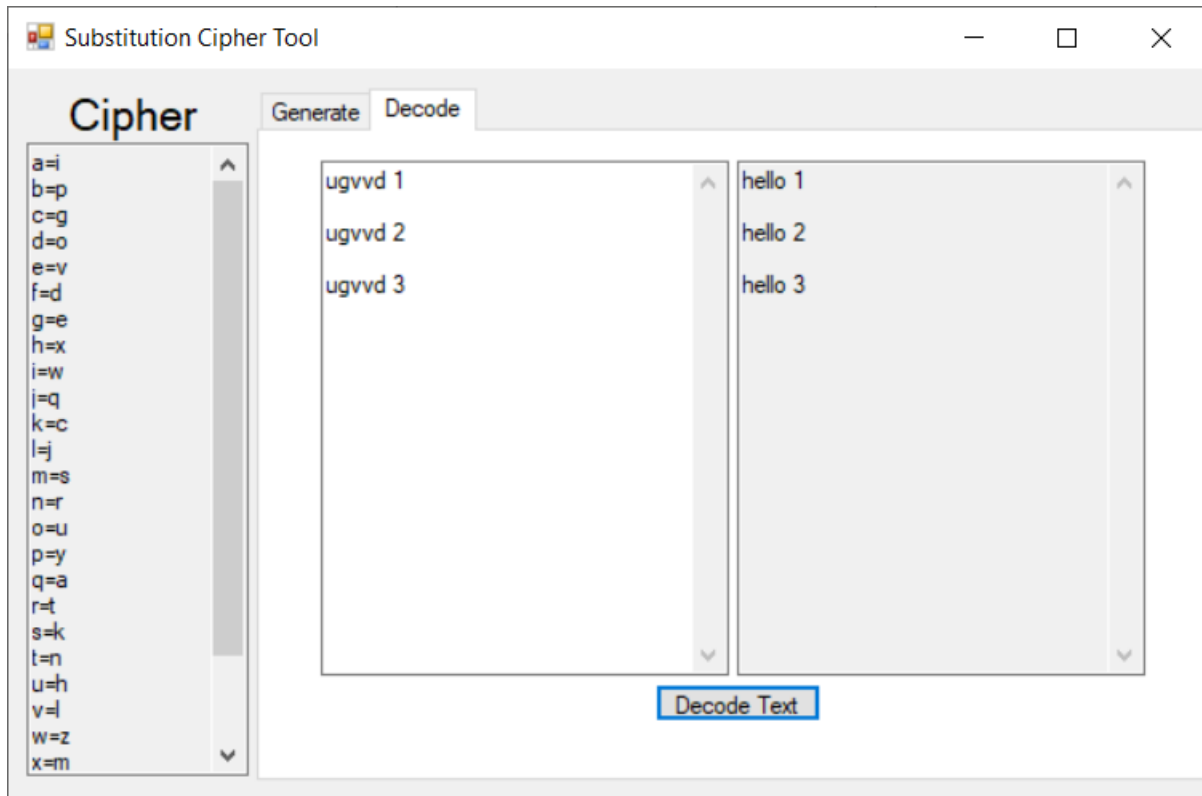
4: What the "Generate" tab should look like after clicking the "Generate Cipher" button.



5: What the "Cipher" text box on the left should look like when the "Load Cipher" button is clicked. Notice the cipher in the left text box matches the cipher in the "Generate" tab



6: An example of what SubstitutionCipher.txt should look like after clicking the "Save to File" button.



7: An example of what the "Decode" tab should look like after clicking the "Decode Text" button.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should not appear on any work you submit for grading.
Remember to use your contestant # where appropriate.

Submit a copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).

Your application will be scored on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts
The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to USB drive and program runs from USB _____ 20 pts

If the program does not execute, then the remaining items in the program executive section receive a score of zero.

The title of the form is “Substitution Cipher Tool” _____ 10 pts
The main form does not have minimize or maximize buttons
in the Form Handle _____ 10 pts
The form’s start position is the center of the screen and it cannot be resized _____ 10 pts
The window has the correct buttons, text boxes, radio buttons and
labels based upon the Figures _____ 20 pts
Every text box has a vertical scrollbar to handle long inputs _____ 20 pts
The program reads all lines from the EncodedTexts.txt file _____ 20 pts
The program correctly reads in the cipher key from the SubstitutionKey.txt file _____ 20 pts
The program correctly displays the cipher key in its text box on startup _____ 20 pts
The program correctly displays the encoded texts in its text box on startup _____ 20 pts
The program correctly displays the decoded texts in its text box on startup _____ 20 pts
Clicking “Generate Cipher” correctly generates a full random cipher key _____ 10 pts
Clicking “Generate Cipher” loads the random cipher key into its text box _____ 10 pts
Clicking “Load Cipher” correctly replaces the current cipher key with the last generated cipher
key _____ 20 pts
Clicking “Load Cipher” loads the last generated cipher key into its text box _____ 10 pts
Clicking “Load Cipher” does nothing if a cipher key has not been generated _____ 10 pts

Clicking “Save to File” correctly overwrites the contents of SubstitutionKey.txt with the
serialized current cipher key _____ 20 pts
Clicking “Save to File” does nothing if a cipher key has not been generated _____ 10 pts
Clicking “Decode” decrypts each encoded text string using the loaded cipher key _____ 10 pts
Clicking “Decode” loads each decoded text string into the decoded texts text box _____ 10 pts
The decryption ignores non-alphabetical characters and converts upper-case letters to lower-case
letters _____ 20 pts

Source Code Review

Code is commented at the top, for each function, and as needed _____ 10 pts
Code uses reasonable and consistent variable naming conventions _____ 10 pts
Code for “GenerateCipherButton_Click” method is present _____ 50 pts
Code for “LoadCipherButton_Click” method is present _____ 30 pts
Code for “SaveToFileButton_Click” method is present _____ 50 pts
Code for “DecodeTextButton_Click” method is present _____ 50 pts
Code for reading the cipher key from text file is present _____ 10 pts
Code for reading the encoded texts from text file is present _____ 10 pts
Code for error handling text file parsing errors is present _____ 20 pts

Total Points: _____ / 530 pts

```
// CS_330_01_2345_6789
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace CSharpProgramming_Nationals
{
    public partial class SubstitutionCipherForm : Form
    {
        private const string Alphabet = "abcdefghijklmnopqrstuvwxyz";
        private const string SubstitutionKeyFilePath = "../SubstitutionKey.txt";
        private const string EncodedTextsFilePath = "../EncodedTexts.txt";

        private readonly Random RandomNumberGenerator;
        private IDictionary<char, char> SubstitutionKey;
        private IDictionary<char, char> GeneratedSubstitutionKey;
        private string[] EncodedTexts;

        public SubstitutionCipherForm()
        {
            InitializeComponent();
            RandomNumberGenerator = new Random();
            SubstitutionKey = LoadSubstitutionKeyFromFile();
            EncodedTexts = LoadEncodedTextsFromFile();
            LoadDataIntoForm();
        }
    }
}
```

```
private IDictionary<char, char> LoadSubstitutionKeyFromFile()
{
    var cipherKeyFile = File.ReadAllText(SubstitutionKeyFilePath);
    var substitutionKey = new Dictionary<char, char>();

    if (cipherKeyFile.Length != Alphabet.Length)
    {
        // Show some error
    }

    for (var i = 0; i < cipherKeyFile.Length; i++)
    {
        substitutionKey[Alphabet[i]] = cipherKeyFile[i];
    }

    return ConvertKeyStringToMap(cipherKeyFile);
}

private string[] LoadEncodedTextsFromFile()
{
    var encodedTextsFile = File.ReadAllText(EncodedTextsFilePath);
    var separatedEncodedTexts = encodedTextsFile.Split(';');

    return separatedEncodedTexts;
}

private void LoadDataIntoForm()
{
    LoadSubstitutionKeyIntoForm(SubstitutionKey, SubstitutionKeyTextBox);
    LoadEncodedTextsIntoForm(EncodedTexts, EncodedTextTextBox);
}

private void LoadSubstitutionKeyIntoForm(IDictionary<char, char> substitutionKey,
TextBox textBox)
{
    var stringBuilder = new StringBuilder();
    foreach (var alphabetCharacter in Alphabet)
    {
        var formattedSubKey =
        $"{alphabetCharacter}={substitutionKey[alphabetCharacter]}{Environment.NewLine}";
        stringBuilder.Append(formattedSubKey);
    }
    textBox.Text = stringBuilder.ToString();
}

private void LoadEncodedTextsIntoForm(string[] encodedTexts, TextBox textBox)
{
    var stringBuilder = new StringBuilder();
    foreach (var text in encodedTexts)
    {
        var formattedText = $"{text}{Environment.NewLine}{Environment.NewLine}";
        stringBuilder.Append(formattedText);
    }
    textBox.Text = stringBuilder.ToString();
}

private IDictionary<char, char> GenerateSubstitutionKey()
{
    var stringBuilder = new StringBuilder(Alphabet.Length);
}
```

```
        var alphabetCharacters = Alphabet.ToList();
        for (var i = 0; i < Alphabet.Length; i++)
        {
            var randomIndex = RandomNumberGenerator.Next(0,
alphabetCharacters.Count);
            stringBuilder.Append(alphabetCharacters[randomIndex]);
            alphabetCharacters.RemoveAt(randomIndex);
        }

        return ConvertKeyStringToMap(stringBuilder.ToString());
    }

    private string DecodeText(string encodedText)
    {
        var stringBuilder = new StringBuilder(encodedText.Length);

        foreach (var encodedCharacter in encodedText)
        {
            var lowercasedEncodedCharacter = char.ToLower(encodedCharacter);
            var decodedCharacter = char.IsLetter(lowercasedEncodedCharacter) ?
                SubstitutionKey[lowercasedEncodedCharacter] :
                lowercasedEncodedCharacter;

            stringBuilder.Append(decodedCharacter);
        }

        return stringBuilder.ToString();
    }

    private void GenerateCipherButton_Click(object sender, EventArgs e)
    {
        var generatedKey = GenerateSubstitutionKey();
        GeneratedSubstitutionKey = generatedKey;
        LoadSubstitutionKeyIntoForm(generatedKey, GeneratedKeyTextBox);
    }

    private void LoadCipherButton_Click(object sender, EventArgs e)
    {
        SubstitutionKey = GeneratedSubstitutionKey;
        LoadSubstitutionKeyIntoForm(GeneratedSubstitutionKey,
SubstitutionKeyTextBox);
    }

    private void SaveToFileButton_Click(object sender, EventArgs e)
    {
        if (GeneratedSubstitutionKey == null)
        {
            //Show some error message
            return;
        }

        var substitutionKeyString = ConvertMapToKeyString(GeneratedSubstitutionKey);
        File.WriteAllText(SubstitutionKeyFilePath, substitutionKeyString);
    }

    private IDictionary<char, char> ConvertKeyStringToMap(string substitutionKey)
    {
        var substitutionKeyMap = new Dictionary<char, char>();
        for (var i = 0; i < substitutionKey.Length; i++)
```

```
        {
            substitutionKeyMap[Alphabet[i]] = substitutionKey[i];
        }
        return substitutionKeyMap;
    }

    private string ConvertMapToKeyString(IDictionary<char, char> substitutionKeyMap)
    {
        var stringBuilder = new StringBuilder(Alphabet.Length);
        foreach (var alphabetCharacter in Alphabet)
        {
            stringBuilder.Append(substitutionKeyMap[alphabetCharacter]);
        }
        return stringBuilder.ToString();
    }

    private void DecodeTextButton_Click(object sender, EventArgs e)
    {
        var decodedTexts = new List<string>(EncodedTexts.Length);
        foreach (var encodedText in EncodedTexts)
        {
            var decodedText = DecodeText(encodedText);
            decodedTexts.Add(decodedText);
        }

        LoadEncodedTextsIntoForm(decodedTexts.ToArray(), DecodedTextTextBox);
    }
}
```